

Advanced Parallel Programming & Architecture

Laboratory Examination & Silicon Profiling Worksheet: Empirical Microarchitectural Bottlenecks
Session Duration: 120 Minutes | Total Evaluation: 100 Points | Mode: Individual Bare-Metal Execution

Student Full Name	Sultanseitova Nuriya
Student Identification Number	230103360
Date & Laboratory Session	17 September 2026 inf 314
Workstation / Laptop Make & Model	MacBook Air 13-inch (M4, 2025), Mac16,12
Processor SKU / SoC Model	Apple M4 16 GB unified memory
Core Topology (Physical P-cores / E-cores)	10 physical (4 P + 6 E); 10 logical; no SMT
Cache Hierarchy Capacity	L1 P: 192I/128D KiB; E: 128I/64D KiB L2: 16 MiB P + 4 MiB E; L3: not reported
Host OS & Python Runtime	macOS 26.6.2 CPython 3.14.3 arm64 GIL on

■ **MANDATORY LABORATORY PROTOCOL & HARDWARE FIDELITY NOTICE**

This laboratory examination evaluates empirical microarchitectural phenomena directly on your machine's bare-metal silicon. LLMs and AI coding assistants cannot predict the real runtime numbers, thermal throttling curves, or cache eviction latencies of your specific processor SKU. Any submission containing fabricated, perfectly theoretical, or non-empirical data will receive an automatic zero. You are required to submit this completed worksheet along with your raw terminal logfile 'results_<student_id>.txt' before the 120-minute timer elapses.

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Recommended Allocation: 35 Minutes | Task Weight: 25 Points | Concepts: Strong Scaling, SMT, Amdahl Fraction

Amdahl's Law defines the theoretical upper bound of speedup for a fixed-size workload when scaling across multiple processors:

$$S(p) = 1 / [(1 - P) + (P / p)]$$

where $S(p)$ is the speedup ratio $T(1)/T(p)$, P is the strictly parallelizable fraction of the algorithm, and p is the number of active worker cores. In physical silicon, speedup curves diverge sharply from theoretical models due to process initialization overhead (fork/spawn latency), inter-process communication (IPC) serialization, OS context switching, and resource contention on Simultaneous Multithreading (SMT/Hyper-Threading) logical cores.

■ **EXECUTION PROTOCOL**

Create a script named 'task1_amdahl.py' using the benchmark code below. Execute the script from your terminal: 'python3 task1_amdahl.py'. Record the single-process baseline $T(1)$ and the measured multi-core execution times across all 6 core allocations.

```

import time
import os
import multiprocessing as mp

def cpu_intensive(chunk_size):
    acc = 0
    for i in range(chunk_size):
        acc += (i % 7) * (i % 11)
    return acc

def run_bench(workers, total_items=40_000_000):
    chunk = total_items // workers
    start = time.perf_counter()
    with mp.Pool(processes=workers) as pool:
        pool.map(cpu_intensive, [chunk] * workers)
    return time.perf_counter() - start

if __name__ == "__main__":
    cores = [1, 2, 4, 8, 12, 16]
    base = run_bench(1)
    print(f"OS Reported Logical Cores: {os.cpu_count()}")
    print(f"Single Process Baseline Time T(1): {base:.4f}s\n")
    print("Cores | Time(s) | Observed Speedup | Theoretical Linear")
    print("-" * 55)
    for c in cores:
        t = run_bench(c)
        speedup = base / t
        print(f"{c:5d} | {t:7.4f} | {speedup:16.2f}x | {c:18.2f}x")

```

p = 1 (Baseline)	1.37100892	1.00x	1.00x	100.0%
p = 2	0.59067937	2.321071x	2.00x	116.05%
p = 4	0.35399267	3.872987x	4.00x	96.82%
p = 8	0.29870246	4.589882x	8.00x	57.37%
p = 12	0.30245396	4.532951x	12.00x	37.77%
p = 16	0.34560379	3.966996x	16.00x	24.79%

[Q1.1] Identify the exact inflection point (core count p^*) where your observed speedup curve flattens or degrades. Correlate this directly to your machine's physical hardware topology (distinguish between physical Performance cores,

Efficiency cores, and logical SMT hyperthreads). (8 Points)

Student Response & Architectural Justification:

Best sampled count: $p^*=8$ (4.5899x). First sampled decline: $p=12$.

~~Explanation continues at the top of the next page.~~

Sultanseitova Nuriya | 230103360 | 2/13

Q1.1 continued: 8 workers take 0.29870246 s; 12 take 0.30245396 s (1.26% slower), and 16 take 0.34560379 s (15.70% slower than 8). The M4 has 4 P-cores and 6 slower E-cores, with no SMT. Gains diminish beyond four workers. At 12/16 workers, processes outnumber all 10 cores; scheduling and startup can outweigh concurrency. The small 8-to-12 change may include noise. Counts 9-11 were not sampled, so an exact hardware threshold cannot be inferred.

Baseline: $T(1)=1.37100892$ s; the later $p=1$ sweep was 1.11901204 s (18.38% lower). All ratios use the original baseline; this variability explains why $p=2$ appears superlinear.

[Q1.2] Using your empirical measurements for $T(1)$ and $T(4)$, calculate the exact value of P (the parallel fraction) of this workload on your silicon. Show the complete algebraic derivation: $S(4) = T(1)/T(4) = 1 / [(1 - P) + P/4]$. (9 Points)

Student Response & Architectural Justification:

$S(4) = T(1)/T(4) = 1.37100892 / 0.35399267 = 3.87298675$.

$1/S(4) = (1-P) + P/4 = 1 - 3P/4$.

$3P/4 = 1 - 1/S(4)$. Therefore $P = (4/3)(1 - 1/S(4))$.

$P = 0.98906845 = 98.9068\%$; effective serial fraction = 1.0932%.

This is a two-point effective fit including runtime/startup costs, not an exact intrinsic fraction of the algorithm.

Baseline variability and unequal core speeds limit the fit.

[Q1.3] Compute the Asymptotic Speedup Limit $S_{\max} = \lim(p \rightarrow \infty) S(p) = 1 / (1 - P)$ using your calculated P from Q1.2. Explain what this limit implies if your department purchased a 128-core server node to run this specific workload. (8 Points)

Student Response & Architectural Justification:

As p tends to infinity, P/p tends to 0, so $S_{\max} = 1/(1-P)$.

$S_{\max} = 1/0.01093155 = 91.4783x$.

Under the same fitted assumptions, $S(128) = 1/[(1-P)+P/128] = 53.5945x$, not 128x. Serial/overhead costs limit the benefit of extra cores. A different server would need new measurements because its core speeds, startup costs and memory system would change.

TASK 2: CACHE COHERENCE & THE FALSE SHARING TRAP

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: MESI Protocol, 64-Byte Cache Lines, Bus Invalidation

Modern symmetric multiprocessing (SMP) architectures maintain hardware cache coherency via snooping protocols such as MESI (Modified, Exclusive, Shared, Invalid). Data moves between main memory (DRAM) and CPU caches in discrete 64-byte blocks called cache lines. False Sharing occurs when two independent threads executing on physically separate CPU cores continuously write to distinct variables that happen to reside on the same 64-byte cache line. Although no logical data race exists, each write forces the core's cache controller to broadcast an Invalidation signal across the interconnect bus, evicting the line from the other core's L1/L2 cache and causing massive pipeline stalls.

■ EXECUTION PROTOCOL

Create 'task2_falsesharing.py'. Run the benchmark 3 separate times to eliminate background OS scheduling noise. The test compares 4 threads writing to adjacent list indices vs. 4 threads writing to padded indices separated by 16 elements ($16 * 8$ bytes = 128 bytes, guaranteeing placement on distinct 64-byte cache lines). Record the median execution times.

Sultanseitova Nuriya | 230103360 | 3/13

```
import threading
import time

ITERATIONS = 5_000_000

def worker_adjacent(shared_list, index):
    for _ in range(ITERATIONS):
        shared_list[index] += 1

def worker_padded(shared_list, index):
    # Offset by 16 integers (16 * 8 bytes = 128 bytes > 64-byte cache line)
    padded_idx = index * 16
    for _ in range(ITERATIONS):
        shared_list[padded_idx] += 1

def run_test(target_fn, size):
    arr = [0] * size
    threads = [threading.Thread(target=target_fn, args=(arr, i)) for i in range(4)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return time.perf_counter() - start

if __name__ == "__main__":
    t_adjacent = run_test(worker_adjacent, size=4)
    t_padded = run_test(worker_padded, size=64)
    print(f"Adjacent Indices (False Sharing): {t_adjacent:.4f}s")
    print(f"Padded Indices (Cache-Aligned): {t_padded:.4f}s")
    print(f"Slowdown Factor: {t_adjacent / t_padded:.2f}x")
```

Adjacent Indices (False Sharing Active)	0.38956883	0.36908788	0.36997200	0.36997200	— [Baseline]
Padded Indices (Cache-Line Aligned)	0.36691042	0.37141242	0.37171654	0.37141242	1.003893x

[Q2.1] Calculate the exact memory footprint of a 64-bit integer pointer/reference in CPython. Mathematically prove why

an index stride of 16 guarantees that adjacent threads never access memory within the same 64-byte L1 cache line. (8 Points)

Student Response & Architectural Justification:

A 64-bit reference is $64/8 = 8$ bytes (confirmed by `struct.calcsize`). List slot i is at $A+8i$, so slots $16j$ and $16(j+1)$ are 128 bytes apart. Their line numbers differ by 2 for 64-byte lines and by 1 for this Mac's reported 128-byte lines: $\text{floor}((A+128j)/L) = \text{floor}(A/L) + (128/L)j$. Naturally aligned 8-byte pointer slots do not straddle those boundaries. This proves separation of pointer slots, not of all heap objects touched: Python integers are separate objects. The padded/adjacent median time is 1.003893x; adjacent/padded is 0.996122x. No consistent benefit was observed. The GIL is enabled, so this Python-thread test does not isolate simultaneous cross-core false sharing.

Sultanseitova Nuriya | 230103360 | 4/13

[Q2.2] Trace the MESI state transitions for the shared cache line containing array elements. When Core 0 executes an increment on index 0 and Core 1 subsequently executes an increment on index 1, describe the bus invalidation messages and state shifts (Shared -> Modified -> Invalid). (9 Points)

Student Response & Architectural Justification:

Assume both cores initially hold the line in Shared (S). Core 0 requests write ownership (BusUpgr), invalidates Core 1's copy (S -> I) and changes its own state S -> Modified (M). Core 1 then requests ownership (BusRdX/RFO); Core 0 supplies or flushes dirty data and changes M -> I, while Core 1 gains M and writes. Ownership moves at whole-line granularity although indices 0 and 1 are independent. If Core 1 reads first, BusRd can cause M -> S and S/S before a later upgrade. This is the requested conceptual MESI trace, not an observed trace or proof of Apple's exact coherence protocol.

[Q2.3] In compiled systems programming, manual array padding wastes memory. Describe two industry-standard compile-time or VM-level mechanisms used to eliminate false sharing (e.g., C++11 'alignas(64)', Java '@Contended', or Go struct field padding). (8 Points)

Student Response & Architectural Justification:

1. C++11: wrap each hot counter in its own aligned type, e.g. `struct alignas(64) Counter { long long value; };`. Its alignment and array stride separate counters on 64-byte lines. On a 128-byte-line target, use matching 128-byte spacing/alignment. Aligning only the start of a tightly packed array is insufficient.
2. Go: insert explicit blank padding between hot fields, e.g. 120 bytes after an 8-byte counter for 128-byte spacing; verify offsets for the target layout. Both techniques trade memory for less coherence traffic. They do not eliminate the memory cost of padding.

TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: Lock Contention, Race Conditions, Map-Reduce Partitioning

Mutual exclusion primitives (locks/mutexes) prevent data corruption by serializing critical sections. However, coarse grained locks introduce catastrophic performance penalties: user-space thread suspension, OS kernel context switches (futex system calls), and cache line invalidation across cores competing for the lock variable. In distributed and concurrent systems architecture, the foundational principle is: 'The fastest lock is no lock.'

■ EXECUTION PROTOCOL

Create 'task3_sync.py'. Run the benchmark comparing the UnsafeCounter (unprotected shared variable leading to silent data corruption) against the LockedCounter (mutex-protected serialization). Record the corrupted value and the elapsed runtimes.

```
import threading
import time
```

```
TOTAL_OPS = 2_000_000
NUM_THREADS = 4
```

Sultanseitova Nuriya | 230103360 | 5/13

```
class UnsafeCounter:
    def __init__(self): self.val = 0
    def inc(self): self.val += 1

class LockedCounter:
    def __init__(self):
        self.val = 0
        self.lock = threading.Lock()
    def inc(self):
        with self.lock:
            self.val += 1

def bench(counter_type):
    c = counter_type()
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    def work():
        for _ in range(ops_per_thread):
            c.inc()
    threads = [threading.Thread(target=work) for _ in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return c.val, time.perf_counter() - start

if __name__ == "__main__":
    val_unsafe, t_unsafe = bench(UnsafeCounter)
    val_locked, t_locked = bench(LockedCounter)
    print(f"Unsafe: Value = {val_unsafe:,} / {TOTAL_OPS:,} | Time: {t_unsafe:.4f}s")
    print(f"Locked: Value = {val_locked:,} / {TOTAL_OPS:,} | Time: {t_locked:.4f}s")
    print(f"Contention Cost Multiplier: {t_locked / t_unsafe:.2f}x")
```

Unsafe Accumulator (Race Condition)	2,000,000	2,000,000	0	0.05760088	1.00x [Baseline]
Locked Accumulator (threading.Lock)	2,000,000	2,000,000	0 (Protected)	0.16397437	2.846734x

[Q3.1] Analyze your Unsafe Accumulator output. What was your exact corrupted count? Explain the low-level read modify-write machine code sequence (LOAD, ADD, STORE) that caused lost updates between overlapping thread executions. (7 Points)

Student Response & Architectural Justification:

Observed unsafe value: 2,000,000; lost updates: $2,000,000 - 2,000,000 = 0$. No corruption occurred in this run. A possible lost-update interleaving is: A LOADs 10; B LOADs 10; both ADD 1; both STORE 11. Two increments then produce only one net increment. This is a conceptual read-modify-write sequence, not a disassembly of Python. GIL-enabled CPython and its scheduling can produce the correct result here; one successful run does not prove portable thread safety.

Sultanseitova Nuriya | 230103360 | 6/13

■ ARCHITECTURAL REDESIGN CHALLENGE: LOCKLESS PARTITIONING

Redesign the accumulation algorithm without using any Mutex locks. Requirements:

1. Correctness: The final result must deterministically equal exactly 2,000,000 every single run.
2. Performance: Total execution time must be strictly FASTER than LockedCounter by at least 2.0x. Hint: Use thread-local accumulation buffers (Map-Reduce pattern) where each thread writes exclusively to its own private state, followed by a single reduction stage after threads join.

[Q3.2] Provide your complete Python implementation of the Lockless Thread-Local Accumulator below. Record your measured execution time and calculate the speedup factor achieved over LockedCounter. (11 Points)

Student Response & Architectural Justification:

```
import threading, time
TOTAL_OPS, NUM_THREADS = 2_000_000, 4

def bench_local():
    partials = [0] * NUM_THREADS
    def work(i, n):
        local = 0
        for _ in range(n): local += 1
        partials[i] = local
    q, r = divmod(TOTAL_OPS, NUM_THREADS)
    threads = [threading.Thread(target=work, args=(i, q+(i<r)))]
    for i in range(NUM_THREADS):
        start = time.perf_counter()
        for t in threads: t.start()
        for t in threads: t.join()
        value = sum(partials)
        elapsed = time.perf_counter() - start
        assert value == TOTAL_OPS
        return value, elapsed

if __name__ == "__main__": print(bench_local())
```

Actual sum: 2,000,000 | Time: 0.02502483 s | $0.16397437/0.02502483 = 6.55246632x (>2x)$.

[Q3.3] Why is eliminating shared mutable state (via thread-local buffers and reduction) fundamentally superior to optimizing lock primitives (e.g., adaptive spinlocks or read-write locks) in high-throughput concurrent systems? (7 Points)

Student Response & Architectural Justification:

Each thread performs 500,000 private increments and publishes to its own slot once; after all joins, reduction sums four completed partials to exactly 2,000,000. No mutex is used in the hot loop. This reduces shared writes and repeated serialization; reduction costs $O(\text{threads})$. Tuning a lock still leaves a shared bottleneck. The gain also includes avoiding the original per-increment method call, so it is not a pure mutex-cost measurement. Runtime GIL/join synchronization remains. This design suits associative aggregation; not all shared-state problems can use it.

TASK 4: THE MEMORY WALL VS. COMPUTE SATURATION

Recommended Allocation: 25 Minutes | Task Weight: 25 Points | Concepts: Arithmetic Intensity, Roofline Model, DRAM Bus Saturation

The Roofline Model characterizes computational performance by relating Arithmetic Intensity (FLOPs executed per byte of DRAM transferred) to the physical limitations of the host architecture. When a workload has high arithmetic intensity, execution speed is bound by the CPU's ALU/FPU pipelines (Compute-Bound). When arithmetic intensity is low, multiple

Sultanseitova Nuriya | 230103360 | 7/13

cores saturate the off-chip memory bus and memory controllers (Memory-Bound). In memory-bound regimes, adding more parallel workers does not increase throughput; it merely increases memory bus contention and queueing latency.

■ EXECUTION PROTOCOL

Create 'task4_roofline.py'. Execute both benchmark suites across 1, 2, and 4 worker processes. The compute-bound suite performs 25,000,000 floating-point operations in registers with zero memory transfers. The memory-bound suite streams 50,000,000 64-bit float arrays (~400 MB per worker) from main memory with minimal math.

```

import time
import multiprocessing as mp
import numpy as np

def compute_heavy(n):
    x = 1.0001
    for _ in range(n):
        x = (x * 1.000001) + 0.00001
    return x

def memory_heavy(size):
    arr = np.ones(size, dtype=np.float64)
    arr = arr * 2.0 + 1.0
    return arr[0]

def run_suite():
    print("=== Compute-Bound Suite (Register Math) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(compute_heavy, [25_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

    print("\n=== Memory-Bound Suite (DRAM Bandwidth Saturation) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(memory_heavy, [50_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

if __name__ == "__main__":
    run_suite()

```

w = 1 Worker (Baseline)	0.48374733	1.00x	0.22984987	1.00x
w = 2 Workers	0.48041433	0.993110x	0.28037425	1.219815x

Sultanseitova Nuriya | 230103360 | 8/13

w = 4 Workers	0.49870058	1.030911x	0.66761083	2.904552x

[Q4.1] Analyze the scaling divergence between the two workloads. Why does running 4 parallel compute tasks complete

in roughly the same elapsed time as 1 task, whereas running 4 parallel memory tasks causes total execution time to spike dramatically? (9 Points)

Student Response & Architectural Justification:

Four compute workers take 1.030915x the single-worker time, while four memory workers take 2.904551x. Each worker receives the full workload, so total work increases with workers (weak scaling). Relative aggregate throughput is $4/(T_w/T_1)$: 3.8801x compute and 1.3771x memory. Independent computation uses multiple cores; arrays compete for shared memory resources. However, allocation, page faults, process startup/imports and temporary arrays are also timed, so this does not prove peak DRAM saturation. Each source array is 400 MB plus temporaries. The Python arithmetic loop includes interpreter/object overhead and is not literally zero-memory register-only computation.

[Q4.2] Investigate your host machine's physical memory specifications (e.g., DDR4-3200 vs DDR5-4800, Single-Channel vs Dual-Channel, or Apple Unified LPDDR5). Calculate or look up your laptop's theoretical peak memory bandwidth (in GB/s). Explain how memory bus width limits multi-core scalability. (8 Points)

Student Response & Architectural Justification:

The machine has 16 GB unified memory. Apple specifies 120 GB/s memory bandwidth for the M4 MacBook Air (source on page 13). This is the manufacturer's theoretical peak, not measured application throughput. The supplied hardware log does not expose memory transfer rate, bus width or channel configuration; those values are not invented. Bandwidth (GB/s) = transfer rate (GT/s) x aggregate bus width (bits) / 8. Bus width limits bytes transferred per cycle; adding cores does not widen the interface. Once sustainable shared bandwidth is saturated, more workers divide that bandwidth and increase queueing.

[Q4.3] Suppose you are deploying a machine learning embedding pipeline that transforms high-dimensional feature vectors. Based on your Task 4 findings, would optimizing your code with CUDA GPU kernels or CPU OpenMP pragmas yield greater speedup if your workload is memory-bandwidth saturated? Justify your architectural choice. (8 Points)

Student Response & Architectural Justification:

For a supported discrete GPU with much higher device-memory bandwidth, a fused CUDA pipeline could outperform more CPU OpenMP threads if the data remain on the GPU and transfers are amortized. More CPU threads alone cannot overcome an already saturated memory bus. Reduce data movement and fuse operations first. On this Mac, the GPU is Apple hardware: CUDA is not its backend; Metal provides GPU compute. CPU and GPU share unified memory, so a GPU does not automatically increase DRAM bandwidth. No CUDA, Metal or OpenMP test was run; these results do not establish a numerical GPU speedup or an unconditional winner.

EVALUATION RUBRIC & INSTRUCTOR VERIFICATION SIGN-OFF

Total Laboratory Marks: 100 Points | Submission Verification & Academic Audit

Task 1: Amdahl Scaling	Complete empirical core sweep table, valid inflection identification, rigorous algebraic derivation of P, and asymptotic limit analysis.	25	
Task 2: Cache Coherence	Three-run trial data for adjacent vs padded layouts, mathematical verification of 64-byte cache boundary, and	25	

	MESI protocol trace.		
Task 3: Sync & Lockless	Accurate race condition defect capture, lock contention measurement, functioning lockless map-reduce redesign with >2x verified speedup.	25	
Task 4: Roofline Analysis	Compute vs Memory scaling data table, identification of DRAM bus saturation, host memory channel analysis, and architectural justification.	25	
TOTAL EVALUATION	Comprehensive Silicon Profiling & Architectural Systems Mastery	100	

Instructor / TA Walk-Around Verification Check:	Laboratory Submission Audit Timestamp:
☐ Task 3 Unsafe Corruption Verified ☐ Task 2 Cache Slowdown Verified ☐ Terminal Logfile 'results_<id>.txt' Uploaded	Examiner Signature: _____ Date:

APPENDIX: HARDWARE TOPOLOGY QUERY COMMANDS

Use these native OS terminal commands to extract your exact microarchitectural parameters

Linux (Ubuntu/Debian)	lscpu grep -E 'model name cpu cores' /proc/cpuinfo	lscpu -C sudo dmidecode --type memory grep Speed
macOS (Apple Silicon/Intel)	sysctl -n machdep.cpu.brand_string sysctl hw.perflevel0.physicalcpu	sysctl hw.l1dcachesize hw.l2cachesize system_profiler SPMemoryDataType
Windows (PowerShell / CMD)	Get-CimInstance Win32_Processor Select Name, NumberOfCores wmic cpu get name,NumberOfCores,NumberOfLogicalProcessors	Get-CimInstance Win32_CacheMemory wmic memorychip get speed,capacity,devicelocator

Sultanseitova Nuriya | 230103360 | 10/13

Bonus Task

Copy-paste this code. Do not change the logic. Run it across thread/process counts from 1 to 32, graph your actual execution times, and answer the autopsy questions.

import time

from multiprocessing import Pool

import threading

def cpu_work(n):

Pure CPU crunch

```
count = 0

for i in range(n):

    count += i * i

return count
```

Run both experiments with total_work = 50_000_000 split across T workers # Test T in [1, 2, 4, 8, 16, 32]

Deliverables:

- The Empirical Curve:** Plot \$T\$ (workers) on the X-axis vs. Execution Time on the Y axis for both threading.Thread and multiprocessing.Pool.
- The Inflection Cliff:** Identify the exact number of workers where adding more workers makes the program *slower*.
- The Architectural Autopsy:**
 - Why does Python's threading curve remain virtually flat or get worse, while multiprocessing scales? (Forces them to articulate the GIL).
 - Look up your laptop's CPU model (e.g., Apple M-series, Intel Core i5/i7, AMD Ryzen). How many *physical performance cores* do you have vs. *logical threads*? Pinpoint the exact point on your graph where physical core saturation occurs, and explain the performance drop when \$T > \text{physical cores}\$ using context-switching overhead.

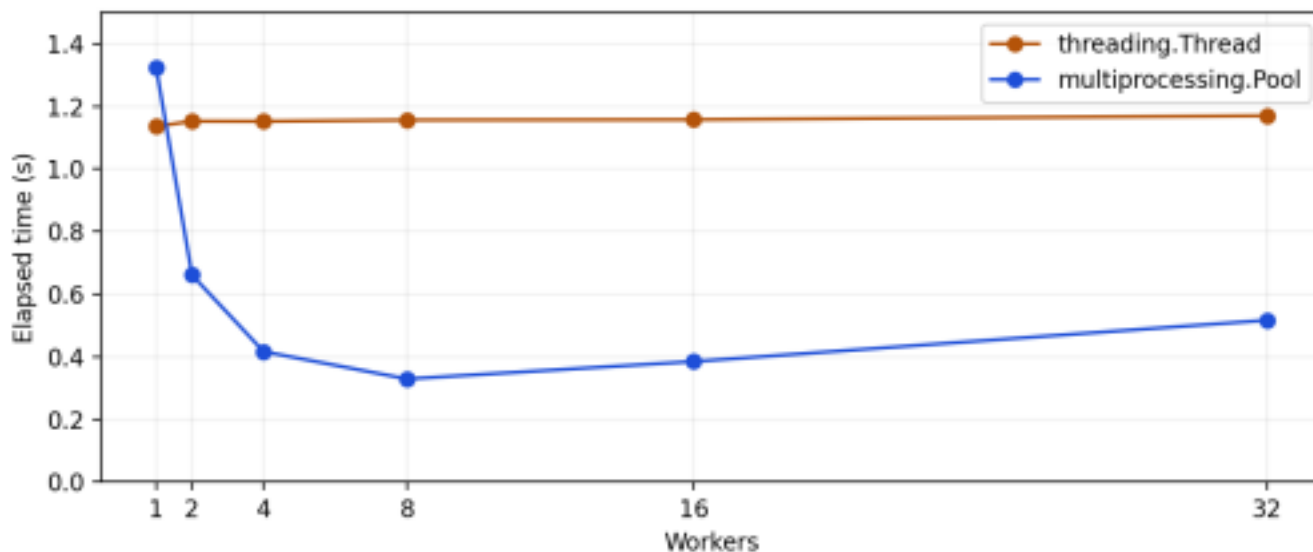
COMPLETED BONUS: see page 12 for the actual thread/process timing table, graph and architectural analysis. Sources and hardware/baseline notes: page 13. Raw evidence: results_230103360.txt and bonus_results_230103360.csv.

Bonus - Threads versus processes

Fixed total work: 50,000,000 loop iterations divided across T workers. The original cpu_work function is unchanged; both implementations receive identical chunks. Times include thread or pool startup and completion. CSV values are rounded below.

Workers	Threads (s)	Processes (s)	Process speedup
1	1.13601267	1.32348171	1.0000x
2	1.15273858	0.66120096	2.0016x
4	1.15245488	0.41666625	3.1764x

8	1.15638383	0.32944988	4.0172x
16	1.15755021	0.38512150	3.4365x
32	1.17015508	0.51689479	2.5604x



Empirical curve and inflection cliff

Threading is almost flat: 1.1360 s at 1 worker and 1.1702 s at 32, about 3.01% slower. Its first sampled increase is at 2 workers, with small fluctuations later. Processes are fastest at 8 workers (0.32944988 s, 4.0172x speedup). The first sampled process regression is at 16 workers (16.90% slower than 8); 32 is 56.90% slower than 8.

Architectural autopsy

The log explicitly reports GIL enabled. CPU-bound Python threads therefore do not execute their Python bytecode in parallel; additional threads add scheduling overhead [2]. Processes use separate interpreters and can run on different cores [5]. The M4 has 4 P-cores and 6 E-cores, totaling 10 physical and 10 logical cores. Gains beyond four workers are not equivalent to adding more P-cores.

The sampled optimum at 8 and degradation at 16 bracket a practical saturation region, but do not pinpoint an exact physical-core saturation threshold. Ten workers were not tested, and affinity was not recorded. At 16 and 32, workers exceed all ten physical cores: scheduling/context switching and additional process startup are plausible costs, together with heterogeneous core speeds and resource contention. Without counters or repeated trials, their individual contributions cannot be separated.

Sultanseitova Nuriya | 230103360 | 12/13

Sources and measurement notes

Student: Sultanseitova Nuriya | ID: 230103360

Raw data: results_230103360.txt and bonus_results_230103360.csv.

Run start: 17 September 2026, 12:48:31 UTC+05. All seven script executions exited successfully. Task 2 was measured three times; other configurations have one recorded measurement. No temperature, affinity or

hardware-counter trace was recorded.

Hardware: L1 per P-core is 192 KiB instruction + 128 KiB data; per E-core it is 128 KiB instruction + 64 KiB data. P-cores share 16 MiB L2 and E-cores share 4 MiB L2. L3 was not reported, not assumed absent. The measured cache-line size is 128 bytes. NumPy version: 2.4.4.

Task 1 uses the separately measured baseline of 1.37100892 s. The later p=1 sweep was 1.11901204 s. The baseline variation limits the Amdahl fit and makes the two-worker speedup exceed 2x. The 12-worker split also drops 4 of 40,000,000 iterations due to integer division, as in the supplied script; this negligible workload difference was retained.

Task 2 table normalizes padded elapsed time to adjacent: 1.003893x. The benchmark's adjacent/padded ratio of medians is 0.996122x. This is no observed slowdown benefit, not an omitted or fabricated measurement.

Task 3 recorded zero unsafe lost updates; the local implementation achieved 6.55246632x speedup. Its code is formatted compactly in the original response space with the same logic as the tested implementation.

Primary reference links:

Apple M4 MacBook Air specifications (120 GB/s):

<https://support.apple.com/en-us/122209>

Python 3.14 threading (GIL):

<https://docs.python.org/3.14/library/threading.html>

Python 3.14 multiprocessing:

<https://docs.python.org/3.14/library/multiprocessing.html>

Microsoft C++ alignas:

<https://learn.microsoft.com/en-us/cpp/cpp/alignas-specifier>

Apple Metal GPU compute:

<https://developer.apple.com/metal/>

Instructor marks, checkboxes and signature remain for the instructor.